

Model Capacity, Regularization and Hyperparameters in PyTorch

1. Lab Objectives:

After completing this lab, students will be able to:

1. Understand the concept of model capacity in neural networks.
2. Identify the difference between underfitting and overfitting.
3. Apply regularization techniques such as dropout and weight decay.
4. Tune hyperparameters to improve model performance.
5. Evaluate models using training and validation metrics.
6. Implement early stopping to prevent overfitting.

2. Introduction:

Deep learning models are capable of learning complex representations from data. However, their performance strongly depends on the design of the neural network and the training configuration.

A key concept in neural network design is model capacity, which determines how complex a function a model can learn.

If the model is too simple, it cannot capture the patterns in the data (underfitting). If the model is too complex, it may memorize the training data (overfitting).

To control this behavior, machine learning practitioners use:

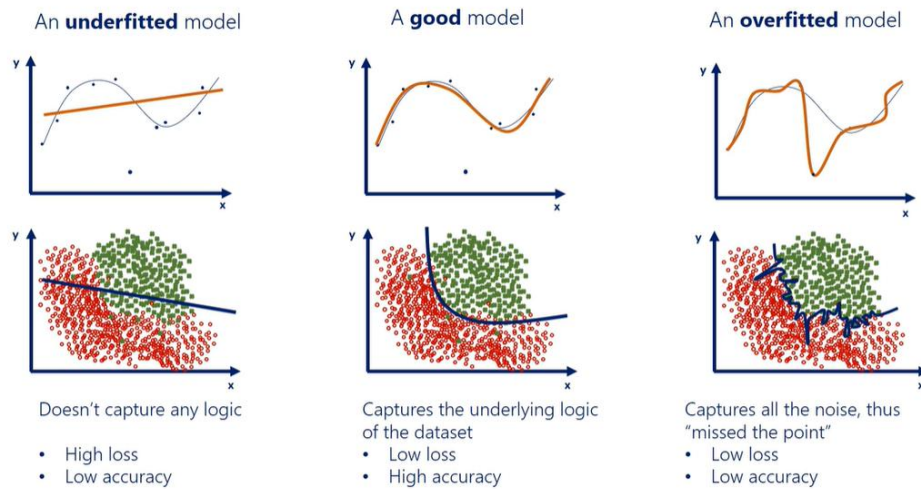
- Regularization techniques
- Hyperparameter tuning
- Early stopping

This lab explores how these elements influence the performance of neural networks implemented using PyTorch.

3. Model Capacity, Underfitting and Overfitting:

Model capacity determines how well a model can approximate complex relationships in the data.

- Low-capacity models → underfitting
- Very high-capacity models → overfitting
- Balanced capacity → best generalization



Bias-variance tradeoff: The balance between underfitting and overfitting

365 DataScience

Scenario	Description
Underfitting	Model too simple to learn
Good Fit	Balanced complexity
Overfitting	Model memorizes training data

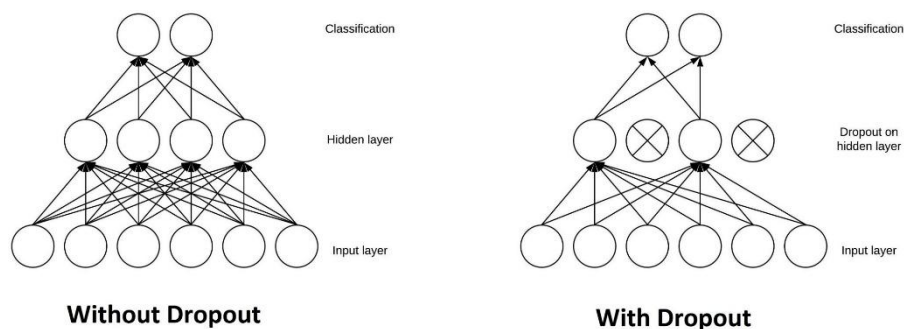
4. Regularization Techniques:

Regularization prevents the model from becoming overly complex and improves generalization.

- **Dropout Regularization:**

Dropout randomly disables neurons during training. This forces the network to learn more robust features. It has following benefits:

- Prevents co-adaptation of neurons
- Improves generalization
- Reduces overfitting



- **Weight Decay (L2 Regularization):**

Weight decay penalizes large weight values. The modified loss function becomes:

$$Loss = Original\ Loss + \lambda \sum w^2$$

This encourages the model to learn smaller, more stable weights.

- **Hyperparameters in Deep Learning:**

Hyperparameters control the learning process. Selecting appropriate hyperparameters significantly improves training efficiency.

Hyperparameter	Role
Learning Rate	Step Size of Gradient Descent
Batch Size	Samples per training iteration
Epochs	Number of training phases
Hidden Units	Neurons in each layer
Network Depth	Number of layers in the network

5. **Dataset:**

In this lab we will use the MNIST Handwritten Digit Dataset.

Property	Value
Classes	10
Image Size	28x28
Training Samples	60,000
Test Samples	10,000

6. **Implementation:**

a. **Import the Libraries:**

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt

✓ 10.6s
```

b. Dataset Loading:

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,),(0.5,))
])

train_dataset = torchvision.datasets.MNIST(
    root='./data',
    train=True,
    download=True,
    transform=transform
)

test_dataset = torchvision.datasets.MNIST(
    root='./data',
    train=False,
    download=True,
    transform=transform
)

train_loader = torch.utils.data.DataLoader(
    train_dataset,
    batch_size=64,
    shuffle=True
)

test_loader = torch.utils.data.DataLoader(
    test_dataset,
    batch_size=64,
    shuffle=False
)
```

c. Low-Capacity Neural Network:

```
class SmallNet(nn.Module):

    def __init__(self):
        super(SmallNet, self).__init__()

        self.fc1 = nn.Linear(784, 32)
        self.fc2 = nn.Linear(32, 10)

    def forward(self, x):

        x = x.view(-1, 784)
        x = torch.relu(self.fc1(x))
        x = self.fc2(x)

        return x
```

✓ 0.0s

```
model = SmallNet().to(device)
print(model)
```

✓ 0.0s

```
SmallNet(
  (fc1): Linear(in_features=784, out_features=32, bias=True)
  (fc2): Linear(in_features=32, out_features=10, bias=True)
)
```

d. High-Capacity Network:

```
class LargeNet(nn.Module):  
    def __init__(self):  
        super(LargeNet, self).__init__()  
  
        self.fc1 = nn.Linear(784, 512)  
        self.fc2 = nn.Linear(512, 256)  
        self.fc3 = nn.Linear(256, 128)  
        self.fc4 = nn.Linear(128, 10)  
  
    def forward(self, x):  
  
        x = x.view(-1, 784)  
  
        x = torch.relu(self.fc1(x))  
        x = torch.relu(self.fc2(x))  
        x = torch.relu(self.fc3(x))  
        x = self.fc4(x)  
  
        return x
```

✓ 0.0s

e. Train the model:

```
criterion = nn.CrossEntropyLoss()  
optimizer = optim.Adam(model.parameters(), lr=0.001)  
train_losses = []  
train_accuracies = []  
epochs = 10  
for epoch in range(epochs):  
    model.train()  
  
    running_loss = 0.0  
    correct = 0  
    total = 0  
  
    for images, labels in train_loader:  
        images, labels = images.to(device), labels.to(device)  
  
        outputs = model(images)  
        loss = criterion(outputs, labels)  
  
        optimizer.zero_grad()  
        loss.backward()  
        optimizer.step()  
  
        running_loss += loss.item()  
  
        _, predicted = torch.max(outputs, 1)  
        total += labels.size(0)  
        correct += (predicted == labels).sum().item()  
  
    epoch_loss = running_loss / len(train_loader)  
    epoch_acc = 100 * correct / total  
  
    train_losses.append(epoch_loss)  
    train_accuracies.append(epoch_acc)  
  
    print(f"Epoch [{epoch+1}/{epochs}], Loss: {epoch_loss:.4f}, Accuracy: {epoch_acc:.2f}%")
```

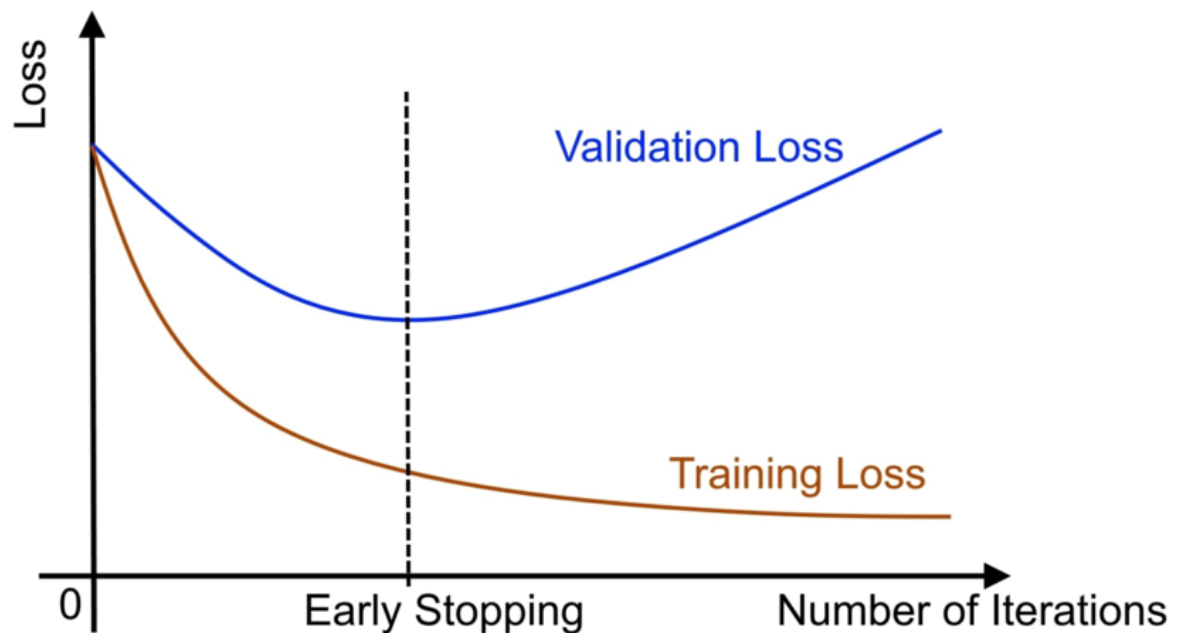
✓ 5.45s

f. Dropout Regularization:

```
class DropoutNet(nn.Module):  
    def __init__(self):  
        super(DropoutNet, self).__init__()  
  
        self.fc1 = nn.Linear(784, 512)  
        self.dropout = nn.Dropout(0.5)  
        self.fc2 = nn.Linear(512, 10)  
  
    def forward(self, x):  
  
        x = x.view(-1, 784)  
  
        x = torch.relu(self.fc1(x))  
        x = self.dropout(x)  
        x = self.fc2(x)  
  
        return x
```

g. Early Stopping:

Early stopping stops training when validation performance stops improving.



Lab Tasks:

1. Load the FashionMNIST dataset using PyTorch. Create training and testing dataloaders with batch size = 64. Display 6 sample images from the dataset. Print the shape of the images and labels. Identify the class name of each displayed image.
2. Create a neural network with (input layer = 784 neurons, one hidden layer = 16 neurons, output layer = 10 neurons). Use ReLU activation, CrossEntropyLoss, Adam optimizer). Train the model for (10, 15,20) epochs and record training loss and training accuracy. Finally evaluate the model on the test dataset and plot the training loss and accuracy curves.
3. Now create another neural network with (input layer = 784, hidden layer 1 = 128 neurons, hidden layer 2 = 64 neurons, output layer = 10 neurons). Train the model for (10,15,20) epochs. Evaluate the model for and record the results.
4. Create a neural network with (input layer = 784, hidden layer 1 = 512, hidden layer 2 = 256, hidden layer 3 = 128, hidden layer 4 = 64, hidden layer 5 = 10). Train the model for (10,15,20) epochs. Evaluate the model and record the results.
5. Now for each of the model created above, apply a little modification. For the first network add one dropout layer after hidden layer, for the other two networks add 2-3 dropout layers. And retrain the network and record outputs for accuracy and loss.
6. Now adjust your learning parameters: test for (lr = 0.01, 0.001 and 0.000012, batch sizes to 32, 64, 128). Record outputs for all.
7. Apply early stopping and record the results.
8. Repeat the above processes for MNIST and CIFAR-10 dataset.
9. After all the experiments, you are required to fill the below table. You should also include this table in your lab manuals.

Model	Variation	Epochs	Learning Rate	Batch Size	Training Loss	Test Loss	Training Accuracy	Test Accuracy
Model 1	Base model	10	0.01	32				
		20	0.001	64				
		30	0.000012	128				
	With Dropout	10	0.01	32				
		20	0.001	64				
		30	0.000012	128				
	Early Stopping	10	0.01	32				
		20	0.001	64				
		30	0.000012	128				
Model 2	Base Model	10	0.01	32				
		20	0.001	64				
		30	0.000012	128				
	With Dropout	10	0.01	32				
		20	0.001	64				
		30	0.000012	128				
	Early Stopping	10	0.01	32				
		20	0.001	64				
		30	0.000012	128				
Model 3	Base Model	10	0.01	32				
		20	0.001	64				
		30	0.000012	128				
	With Dropout	10	0.01	32				
		20	0.001	64				

		30	0.000012	128				
	Early Stopping	10	0.01	32				
		20	0.001	64				
		30	0.000012	128				